

Лабораторная работа №11

Представление графов. DFS и BFS

Цель работы:

1. Изучить способы представления графов в памяти компьютера: матрица смежности и список смежности
2. Освоить алгоритмы обхода графа: поиск в глубину (DFS) и поиск в ширину (BFS)
3. Научиться применять DFS и BFS для решения практических задач (поиск компонент связности, кратчайших путей)
4. Проанализировать сложность алгоритмов для разных способов представления графа
5. Сравнить эффективность матрицы смежности и списка смежности

Теоретическое обоснование

1. Основные понятия теории графов

Граф - это совокупность вершин (узлов) и рёбер (связей) между ними.

Основные термины:

Термин	Определение
Вершина (vertex)	Узел графа
Ребро (edge)	Связь между двумя вершинами
Ориентированный граф	Рёбра имеют направление
Неориентированный граф	Рёбра не имеют направления
Взвешенный граф	Рёбра имеют веса (стоимость прохода)
Степень вершины	Количество рёбер, инцидентных вершине
Путь	Последовательность вершин, соединённых рёбрами
Цикл	Путь, начинающийся и заканчивающийся в одной вершине
Компонента связности	Максимальное множество вершин, любые две из которых соединены путём

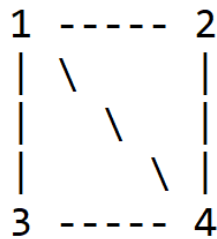
Визуализация графа:

text

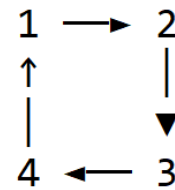
Неориентированный граф:

Ориентированный граф:

Неориентированный граф:



Ориентированный граф:



Вершины: 1, 2, 3, 4

Рёбра: (1,2), (1,3), (1,4), (2,4), (3,4)

Вершины: 1, 2, 3, 4

Рёбра: (1,2), (1,3), (1,4), (2,4), (3,4)

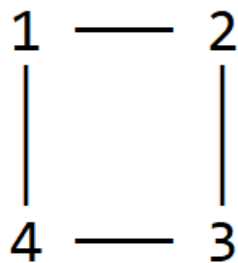
2. Способы представления графов

Матрица смежности (Adjacency Matrix)

Матрица смежности - это квадратная матрица размером $V \times V$, где элемент $matrix[i][j] = 1$, если есть ребро из вершины i в вершину j .

Визуализация:

Граф:



Матрица смежности:

```
[0, 1, 0, 1]
[1, 0, 1, 0]
[0, 1, 0, 1]
[1, 0, 1, 0]
```

```
class GraphMatrix {
private:
    int V, // количество вершин
    vector<vector<int>> adj;

public:
    GraphMatrix(int vertices) : V(vertices) {
        adj.resize(V, vector<int>(V, 0));
    }

    void addEdge(int u, int v) {
        adj[u][v] = 1;
        adj[v][u] = 1; // для неориентированного графа
    }
};
```

```

    }

    void print() {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                cout << adj[i][j] << " ";
            }
            cout << endl;
        }
    }
};

```

Список смежности (Adjacency List)

Список смежности - это массив списков, где для каждой вершины хранится список смежных с ней вершин.

Визуализация:

text

Граф: Список смежности:

1 — 2	0: 1, 3
	1: 0, 2
	2: 1, 3
4 — 3	3: 0, 2

```

class GraphList {
private:
    int V, // количество вершин
    vector<vector<int>> adj;

public:
    GraphList(int vertices) : V(vertices) {
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // для неориентированного графа
    }

    void print() {
        for (int i = 0; i < V; i++) {
            cout << i << ": ";

```

```

    for (int v : adj[i]) {
        cout << v << " ";
    }
    cout << endl;
}
}
};

```

3. Сравнение способов представления

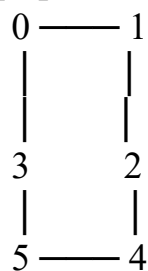
Характеристика	Матрица смежности	Список смежности
Память	$O(V^2)$	$O(V + E)$
Проверка наличия ребра (u,v)	$O(1)$	$O(\deg(u))$
Перебор всех соседей вершины	$O(V)$	$O(\deg(u))$
Добавление ребра	$O(1)$	$O(1)$
Удаление ребра	$O(1)$	$O(\deg(u))$
Когда лучше	Плотные графы ($E \approx V^2$)	Разреженные графы ($E \ll V^2$)

4. Поиск в глубину (DFS - Depth-First Search)

Принцип работы: Идём «в глубину», пока не достигнем тупика, затем возвращаемся и исследуем другие пути.

Визуализация DFS:

Граф:



Порядок обхода DFS (начиная с 0):

$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5$

(если соседи перебираются в порядке возрастания)

Стек вызовов:

push(0) → pop(0) → push(1,3) → pop(3) → push(5) → ...

```
void DFS(int start) {
    vector<bool> visited(V, false);
    DFSUtil(start, visited);
}

void DFSUtil(int v, vector<bool>& visited) {
    visited[v] = true;
    cout << v << " ";

    for (int neighbor : adj[v]) {
        if (!visited[neighbor]) {
            DFSUtil(neighbor, visited);
        }
    }
}
```

Рекурсивная реализация:

```
void DFS_recursive(int v, vector<bool>& visited) {
    visited[v] = true;
    cout << v << " ";

    for (int u : adj[v]) {
        if (!visited[u]) {
            DFS_recursive(u, visited);
        }
    }
}
```

Итеративная реализация (с использованием стека):

```
void DFS_iterative(int start) {
    vector<bool> visited(V, false);
    stack<int> st;

    st.push(start);
    visited[start] = true;

    while (!st.empty()) {
        int v = st.top();
        st.pop();
    }
}
```

```

        cout << v << " ";

    for (int u : adj[v]) {
        if (!visited[u]) {
            visited[u] = true;
            st.push(u);
        }
    }
}
}
}

```

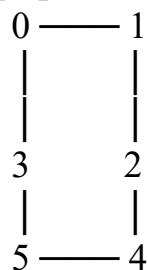
5. Поиск в ширину (BFS - Breadth-First Search)

Принцип работы: Сначала посещаем все соседние вершины, затем переходим к их соседям (по уровням).

Визуализация BFS:

text

Граф:



Порядок обхода BFS (начиная с 0):

0 → 1 → 3 → 2 → 5 → 4
(по уровням)

Очередь:

queue: [0] → pop(0) → push(1,3) → queue: [1,3]

pop(1) → push(2) → queue: [3,2]

pop(3) → push(5) → queue: [2,5]

pop(2) → push(4) → queue: [5,4]

...

```

void BFS(int start) {
    vector<bool> visited(V, false);
    queue<int> q;

```

```

    visited[start] = true;

```

```

    q.push(start);

```

```

while (!q.empty()) {

```

```

    int v = q.front();

```

```

    q.pop();

```

```

    cout << v << " ";

```

```

    for (int u : adj[v]) {
        if (!visited[u]) {
            visited[u] = true;
            q.push(u);
        }
    }
}
}
}

```

6. Применение DFS и BFS

Задача	DFS	BFS
Поиск компонент связности	+	+
Проверка наличия цикла	+	+
Топологическая сортировка	+	-
Поиск кратчайшего пути (невзвешенный граф)	-	+
Поиск мостов и точек сочленения	+	-
Поиск пути (любого)	+	+

7. Поиск компонент связности

```

int countConnectedComponents() {
    vector<bool> visited(V, false);
    int components = 0;

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            components++;
            DFSUtil(i, visited);
        }
    }
    return components;
}

```

8. Поиск кратчайшего пути в невзвешенном графе (BFS)

```

vector<int> shortestPath(int start, int end) {
    vector<int> dist(V, -1);
    vector<int> prev(V, -1);
    queue<int> q;

    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();

        for (int u : adj[v]) {
            if (dist[u] == -1) {
                dist[u] = dist[v] + 1;
                prev[u] = v;
                q.push(u);
            }
        }
    }

    // Восстановление пути
    vector<int> path;
    for (int v = end; v != -1; v = prev[v]) {
        path.push_back(v);
    }
    reverse(path.begin(), path.end());
    return path;
}

```

Примеры выполнения практической части

Пример 1. Создание графа и обходы (базовый уровень)

Задача: Создать граф, реализовать DFS и BFS, вывести порядок обхода.

```

#include <iostream>
#include <vector>
#include <stack>
#include <queue>

```

```

using namespace std;

class Graph {
private:
    int V; // количество вершин
    vector<vector<int>> adj; // список смежности

    void DFS_util(int v, vector<bool>& visited) {
        visited[v] = true;
        cout << v << " ";

        for (int u : adj[v]) {
            if (!visited[u]) {
                DFS_util(u, visited);
            }
        }
    }

public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // неориентированный граф
    }

    void DFS(int start) {
        vector<bool> visited(V, false);
        cout << "DFS (рекурсивный): ";
        DFS_util(start, visited);
        cout << endl;
    }

    void DFS_iterative(int start) {
        vector<bool> visited(V, false);
        stack<int> st;

        st.push(start);
        visited[start] = true;
    }

```

```

cout << "DFS (итеративный): ";
while (!st.empty()) {
    int v = st.top();
    st.pop();
    cout << v << " ";

    for (int u : adj[v]) {
        if (!visited[u]) {
            visited[u] = true;
            st.push(u);
        }
    }
}
cout << endl;
}

```

```

void BFS(int start) {
    vector<bool> visited(V, false);
    queue<int> q

    visited[start] = true;
    q.push(start);

    cout << "BFS: ";
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        cout << v << " ";

        for (int u : adj[v]) {
            if (!visited[u]) {
                visited[u] = true;
                q.push(u);
            }
        }
    }
    cout << endl;
}

```

```

void print() {
    for (int i = 0; i < V; i++) {
        cout << i << ": ";
    }
}

```

```

        for (int u : adj[i]) {
            cout << u << " ";
        }
        cout << endl;
    }
}
};

```

```

int main() {
    Graph g(6);

    g.addEdge(0, 1);
    g.addEdge(0, 3);
    g.addEdge(1, 2);
    g.addEdge(2, 4);
    g.addEdge(3, 5);
    g.addEdge(4, 5);

    cout << "=== Список смежности ===" << endl;
    g.print();

    cout << "\n=== Обход графа (начиная с вершины 0) ===" << endl;
    g.DFS(0);
    g.DFS_iterative(0);
    g.BFS(0);

    return 0;
}

```

Ожидаемый вывод:

=== Список смежности ===

0: 1 3

1: 0 2

2: 1 4

3: 0 5

4: 2 5

5: 3 4

=== Обход графа (начиная с вершины 0) ===

DFS (рекурсивный): 0 1 2 4 5 3

DFS (итеративный): 0 3 5 4 2 1

BFS: 0 1 3 2 5 4

Пример 2. Поиск компонент связности (средний уровень)

Задача: Определить количество компонент связности в графе.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
class Graph {
```

```
private:
```

```
    int V,
```

```
    vector<vector<int>> adj;
```

```
    void DFS(int v, vector<bool>& visited) {
```

```
        visited[v] = true;
```

```
        for (int u : adj[v]) {
```

```
            if (!visited[u]) {
```

```
                DFS(u, visited);
```

```
            }
```

```
        }
```

```
    }
```

```
public:
```

```
    Graph(int vertices) : V(vertices) {
```

```
        adj.resize(V);
```

```
    }
```

```
    void addEdge(int u, int v) {
```

```
        adj[u].push_back(v);
```

```
        adj[v].push_back(u);
```

```
    }
```

```
    int countConnectedComponents() {
```

```
        vector<bool> visited(V, false);
```

```
        int components = 0;
```

```
        for (int i = 0; i < V; i++) {
```

```
            if (!visited[i]) {
```

```
                components++;
```

```
                DFS(i, visited);
```

```
            }
```

```
        }
```

```
        return components;
```

```
    }
```

```

void printComponents() {
    vector<bool> visited(V, false);
    int componentNum = 1;

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            cout << "Компонента " << componentNum++ << ": ";
            vector<int> comp;
            stack<int> st;
            st.push(i);
            visited[i] = true;

            while (!st.empty()) {
                int v = st.top();
                st.pop();
                comp.push_back(v);

                for (int u : adj[v]) {
                    if (!visited[u]) {
                        visited[u] = true;
                        st.push(u);
                    }
                }
            }

            for (int v : comp) {
                cout << v << " ";
            }
            cout << endl;
        }
    }
};

```

```

int main() {
    // Граф с двумя компонентами связности
    Graph g(7);

    // Первая компонента: 0-1-2
    g.addEdge(0, 1);
    g.addEdge(1, 2);
}

```

```

// Вторая компонента: 3-4-5-6
g.addEdge(3, 4);
g.addEdge(4, 5);
g.addEdge(5, 6);

cout << "=== Количество компонент связности: " <<
g.countConnectedComponents() << endl;
g.printComponents();

return 0;
}

```

Ожидаемый вывод:

=== Количество компонент связности: 2

Компонента 1: 0 2 1

Компонента 2: 3 6 5 4

Пример 3. Кратчайший путь в невзвешенном графе (продвинутый уровень)

Задача: Найти кратчайший путь между двумя вершинами с помощью BFS.

```

#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

using namespace std;

class Graph {
private:
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
}

```

```

vector<int> shortestPath(int start, int end) {
    vector<int> dist(V, -1);
    vector<int> prev(V, -1);
    queue<int> q

    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();

        if (v == end) break;

        for (int u : adj[v]) {
            if (dist[u] == -1) {
                dist[u] = dist[v] + 1;
                prev[u] = v;
                q.push(u);
            }
        }
    }

    // Восстановление пути
    vector<int> path;
    if (dist[end] == -1) return path;

    for (int v = end; v != -1; v = prev[v]) {
        path.push_back(v);
    }
    reverse(path.begin(), path.end());

    return path;
}

void printPath(int start, int end) {
    vector<int> path = shortestPath(start, end);
    if (path.empty()) {
        cout << "Путь между " << start << " и " << end << " не существует"
<< endl;
        return;
    }
}

```

```

        cout << "Кратчайший путь от " << start << " до " << end << ": ";
        for (int i = 0; i < path.size(); i++) {
            cout << path[i];
            if (i < path.size() - 1) cout << " → ";
        }
        cout << " (длина: " << path.size() - 1 << ")" << endl;
    }
};

```

```

int main() {
    Graph g(8);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.addEdge(3, 5);
    g.addEdge(4, 5);
    g.addEdge(5, 6);
    g.addEdge(6, 7);

    cout << "=== Поиск кратчайшего пути ===" << endl;
    g.printPath(0, 6);
    g.printPath(0, 7);
    g.printPath(1, 4);
    g.printPath(0, 5);

    return 0;
}

```

Ожидаемый вывод:

=== Поиск кратчайшего пути ===

Кратчайший путь от 0 до 6: 0 → 2 → 4 → 5 → 6 (длина: 4)

Кратчайший путь от 0 до 7: 0 → 2 → 4 → 5 → 6 → 7 (длина: 5)

Кратчайший путь от 1 до 4: 1 → 0 → 2 → 4 (длина: 3)

Кратчайший путь от 0 до 5: 0 → 2 → 4 → 5 (длина: 3)

Индивидуальные задания

Часть 1. Представление графов

Каждый вариант содержит задание реализовать граф с указанным способом представления:

№ вар.	Способ представлени я	Тип графа	Особенности
1	Список смежности	Неориентированн ый	Без весов
2	Матрица смежности	Неориентированный	Без весов
3	Список смежности	Оrientированный	Без весов
4	Матрица смежности	Оrientированный	Без весов
5	Список смежности	Неориентированный	Взвешенный
6	Матрица смежности	Неориентированный	Взвешенный
7	Список смежности	Оrientированный	Взвешенный
8	Матрица смежности	Оrientированный	Взвешенный
9	Список смежности	Неориентированный	Разреженный
10	Матрица смежности	Неориентированный	Плотный
11	Список смежности	Оrientированный	С петлями
12	Матрица смежности	Оrientированный	С петлями
13	Список смежности	Неориентированный	С удалением рёбер

№ вар.	Способ представления	Тип графа	Особенности
14	Матрица смежности	Неориентированный	С удалением рёбер
15	Список смежности + матрица	Неориентированный	Сравнение производительности

Часть 2. Обход графов (DFS и BFS)

Каждый вариант содержит задание реализовать обход графа и выполнить указанную задачу:

№ вар.	Алгоритмы	Задача
1	DFS (рекурсивный), BFS	Подсчитать количество вершин
2	DFS (рекурсивный), BFS	Найти все вершины, достижимые из заданной
3	DFS (рекурсивный), BFS	Найти компоненты связности
4	DFS (рекурсивный), BFS	Проверить, является ли граф связным
5	DFS (рекурсивный), BFS	Найти количество компонент связности
6	DFS (рекурсивный), BFS	Проверить наличие цикла
7	DFS (рекурсивный), BFS	Найти кратчайший путь (BFS)
8	DFS (рекурсивный), BFS	Найти расстояние между двумя вершинами
9	DFS (рекурсивный), BFS	Определить, есть ли путь между вершинами

№ вар.	Алгоритмы	Задача
10	DFS (итеративный), BFS	Вывести вершины в порядке обхода
11	DFS (итеративный), BFS	Найти все вершины на заданном расстоянии
12	DFS (рекурсивный), BFS	Найти диаметр графа (BFS дважды)
13	DFS (рекурсивный), BFS	Найти эйлеров путь
14	DFS (рекурсивный), BFS	Найти количество мостов
15	DFS (рекурсивный), BFS	Найти точки сочленения

Часть 3. Применение обходов

Задание для всех вариантов:

Реализовать программу, которая:

1. Создает граф по заданию (количество вершин и рёбер)
2. Выполняет обход DFS и BFS, начиная с указанной вершины
3. Находит все компоненты связности и выводит их
4. Для выбранной пары вершин находит кратчайший путь (BFS)
5. Выводит результаты в наглядной форме

Пример графа (для тестирования):

text

0 - 1 - 2 - 3

| |

5 - 4 - - 6

Контрольные вопросы

1. Что такое граф? Из каких элементов он состоит?
2. Чем отличается ориентированный граф от неориентированного?
3. Что такое матрица смежности? Каковы её достоинства и недостатки?
4. Что такое список смежности? Каковы его достоинства и недостатки?

5. В каких случаях лучше использовать матрицу смежности, а в каких - список смежности?
6. Какова сложность проверки наличия ребра в матрице смежности? В списке смежности?
7. Какова сложность перебора всех соседей вершины в матрице смежности? В списке смежности?
8. Что такое поиск в глубину (DFS)? Как он работает?
9. Какие задачи решаются с помощью DFS?
10. Что такое поиск в ширину (BFS)? Как он работает?
11. Какие задачи решаются с помощью BFS?
12. В чём отличие рекурсивной реализации DFS от итеративной?
13. Почему BFS находит кратчайший путь в невзвешенном графе, а DFS - нет?
14. Как найти компоненты связности графа с помощью DFS?
15. Как проверить наличие цикла в графе с помощью DFS?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (графы, матрица и список смежности, DFS, BFS)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода, визуализация графа)

Анализ сложности операций для выбранного представления

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Представление графа	25
1.1	Реализация выбранного способа представления	10
1.2	Функции добавления/удаления рёбер	10
1.3	Вывод графа	5

№	Критерий	Баллы
2	Обход графа (DFS и BFS)	35
2.1	Реализация DFS (рекурсивной)	10
2.2	Реализация DFS (итеративной)	5
2.3	Реализация BFS	10
2.4	Корректность обходов на разных графах	10
3	Применение обходов	25
3.1	Поиск компонент связности	10
3.2	Поиск кратчайшего пути (BFS)	10
3.3	Дополнительная задача по варианту	5
4	Оформление и отчёт	15
4.1	Код отформатирован, есть комментарии	5
4.2	Есть ответы на контрольные вопросы	5
4.3	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Некорректная работа DFS (заикливание)	-15
Некорректная работа BFS (порядок обхода)	-10

Нарушение	Штраф
Неверная сложность алгоритма	-10
Нет проверки на пустой граф	-5
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60